AstraNotes Test Improvement Log

CSEN 296B | Week 9 Lab | David Nguyen

Feature Reviewed

I reviewed the course-based sharing feature in AstraNotes. The requirement behind it is FR-5: a user can share a note with a course, and only users enrolled in that course can view it. This is the permission rule that decides who is allowed to open a shared note.

Weak Test Area I Identified

The original test for shared-note access was over-mocked. It patched the permission function itself so the function always returned True, then checked that the route returned 200. The real access rule never ran during the test.

```
from unittest.mock import patch

def test_shared_note_is_viewable(client):
    with patch("astranotes.permissions.can_view", return_value=True):
        response = client.get("/notes/42")
        assert response.status_code == 200
```

What Was Wrong With the Original

The test patched can_view to always return True. That is the exact piece of code I wanted to verify, so the test was checking nothing real. If someone deleted the enrollment check or broke the sharing logic, this test would still pass. It is test theater. It also creates false confidence, because the coverage report counts the route as tested even though the actual access decision was skipped.

Improved Test

The improved test runs the real permission rule against a real in-memory SQLite database. It seeds one enrolled user and one user who is not enrolled, then checks both the allow path and the deny path.

```
def test_course_sharing_respects_enrollment(client, db):
    alice = create_user(db, "alice")
    bob = create_user(db, "bob")
    course = create_course(db, "CSEN296B")
    enroll(db, alice, course)

    note = create_note(
        db,
        author=alice,
        shared_with_course=course,
        body="midterm review notes",
    )

    # enrolled user can see the note
    login(client, alice)
    allowed = client.get(f"/notes/{note.id}")
    assert allowed.status_code == 200
    assert b"midterm review notes" in allowed.data

    # user who is not enrolled is blocked
    login(client, bob)
    denied = client.get(f"/notes/{note.id}")
    assert denied.status_code == 403
```

Alice is enrolled and gets the note. Bob is not enrolled and gets a 403. The test also checks the response body, not just the status code, so it proves the note content actually loads for the allowed user.

Was Mocking Helping or Hiding Risk

In the original test, mocking was hiding risk. It replaced the code under test, so the test could never catch a broken access rule. The permission rule is really a join between course enrollment and note sharing, so mocking it out means the most important logic is never checked. In the improved version I use a real in-memory SQLite database instead of mocking the data. The database and the permission rule stay real because they are the risk. The only thing worth mocking here would be something slow or external, like the email that goes out when a note is shared.

Meaningful Coverage Gap That Still Matters

Even the improved test does not cover access that changes over time. If Bob enrolls in the course later, can he then open the note? If Alice drops the course, does her shared note stay visible to the class or get pulled? The coverage report would mark the route and can_view as covered, but that number does not tell me the enroll and unenroll transitions behave correctly. Hitting a line once does not prove every state of that line is right. That is the gap I would write a test for next.

How AI Helped Critique and Improve the Test

I gave the original test to AI and asked it to find weak spots. It flagged that patching can_view removes the system under test and suggested seeding a real database with one enrolled user and one user who is not enrolled. That matched the over-mock problem I suspected, and it pointed me toward testing the deny path, which the original test ignored.

What I Accepted, Changed, or Rejected

Accepted: the real-database approach and the 403 deny-path assertion. Both made the test check something real.

Changed: the AI version only checked status codes. I added the body assertion so the test proves the note content loads for the allowed user, not just that the route returns 200.

Rejected: the AI also suggested adding many parametrized cases for user roles that AstraNotes does not have yet. That would add tests with no requirement behind them, which is just more test theater. I kept the test small and tied to FR-5.

The new version is stronger because it runs the real rule, checks both the allow and deny paths, asserts on real output, and every assertion maps back to the sharing requirement. That is stronger test signal, not just more tests.